



Explicación Práctica de Semáforos



Ejercicios

SEMÁFOROS - Sintaxis

- Declaraciones de Semáforos

`sem s;` → NO. Si o si se deben inicializar en la declaración

`sem mutex = 1;`

`sem espera[5] = ([5] 1);`

- Operaciones de los Semáforo

$P(s) \rightarrow \langle \text{await } (s > 0) \ s = s-1; \rangle$

$V(s) \rightarrow \langle s = s+1; \rangle$



EJERCICIO 1

Hay C chicos y hay una bolsa con caramelos que nunca se vacía. Los chicos de a UNO van sacando de a UN caramelo y lo comen. Los chicos deben llevar la cuenta de cuantos caramelos se han tomado de la bolsa.

```
int cant = 0;
```

Process Chico[id: 0..C-1]

```
{ while (true)
  { -- tomar caramelo
    cant = cant + 1;
    -- comer caramelo
  }
```

```
int cant = 0;
sem mutex = 1;
```

Process Chico[id: 0..C-1]

```
{ while (true)
  { -- tomar caramelo
    P(mutex);
    cant = cant + 1;
    V(mutex);
    -- comer caramelo
  }
```

```
int cant = 0;
sem mutex = 1;
```

Process Chico[id: 0..C-1]

```
{ while (true)
  { P(mutex);
    -- tomar caramelo
    cant = cant + 1;
    -- comer caramelo
    V(mutex);
  }
```

Se puede
generar una
interferencia

¿Nos asegura que
NO puede haber más
de un chico tomando
un caramelo de la
bolsa?

Reducción de la
conurrencia

EJERCICIO 1

Lo que no pueden hacer al mismo tiempo es trabajar con el recurso compartido (acceder a la bolsa de caramelos e incrementar *cant*), pero SI que más de un chico coma al mismo tiempo → en la solución anterior NO SE MAXIMIZA LA CONCURRENCIA.

```
int cant = 0;
sem mutex = 1;

Process Chico[id: 0..C-1]
{ while (true)
    { P(mutex);
      -- tomar caramelo
      cant = cant + 1;
      V(mutex);
      -- comer caramelo
    }
}
```

En la sección Crítica se debe hacer sólo lo que sea necesario realizar con Exclusión Mutua, el resto debe ir fuera de la SC para maximizar la concurrencia



EJERCICIO 1 - modificado

Para comenzar a tomar caramelos deben esperar a que todos hayan llegado a ese punto. Partimos de la solución anterior.

```
int cant = 0;
sem mutex = 1;
```

Process Chico[id: 0..C-1]

```
{ while (true)
  { P(mutex);
    -- tomar caramelo
    cant = cant + 1;
    V(mutex);
    -- comer caramelo
```

Debemos hacer una barrera entre todos los procesos antes de comenzar con el while.

```
int cant = 0;      sem mutex = 1;
int llegaron = 0; sem barrera = 0;
```

Process Chico[id: 0..C-1]

```
{ P(mutex);
  llegaron ++;
  if (llegaron == C) for i = 1..C {V(barrera);}
  V(mutex);
  P (barrera);
  while (true)
    { P(mutex);
      -- tomar caramelo
      cant = cant + 1;
      V(mutex);
      -- comer caramelo
    }
}
```

EJERCICIO 2

Hay C chicos y hay una bolsa con caramelos **limitada a N caramelos**. Los chicos de a UNO van sacando de a UN caramelo y lo comen. Los chicos deben llevar la cuenta de cuantos caramelos se han tomado de la bolsa.

Comenzamos modificando la solución del Ejercicio 1 para que no intenten sacar más caramelos si la bolsa quedó vacía ($cant = N$)

```
int cant = 0;  
sem mutex = 1;
```

Process Chico[id: 0.. $C-1$]

```
{ while ( $cant < N$ )  
    { P(mutex);  
      -- tomar caramelo  
      cant = cant + 1;  
      V(mutex);  
      -- comer caramelo  
    }  
}
```

Se podrán sacar más de N caramelos.



EJERCICIO 2

El chequeo de la condición que indica que se debe tomar otro caramelo se debe proteger en una SC que también incluya la modificación de esa condición → en este caso el chequeo y $cant = cant + 1$

```
int cant = 0;
sem mutex = 1;
```

Process Chico[id: 0..C-1]

```
{ P(mutex);
  while (cant < N)
  { -- tomar caramelo
    cant = cant + 1;
    -- comer caramelo
  }
  V(mutex);
}
```

Un único chico
tomará los N
caramelos

```
int cant = 0;
sem mutex = 1;
```

Process Chico[id: 0..C-1]

```
{ P(mutex);
  while (cant < N)
  { -- tomar caramelo
    cant = cant + 1;
    V(mutex);
    -- comer caramelo
  }
}
```

Libera la SC y
nunca más
asegura la EM

```
int cant = 0;
sem mutex = 1;
```

Process Chico[id: 0..C-1]

```
{ P(mutex);
  while (cant < N)
  { -- tomar caramelo
    cant = cant + 1;
    V(mutex);
    -- comer caramelo
    P(mutex);
  }
}
```

Sólo un proceso
termina, el resto
se bloquea.

EJERCICIO 2

Al salir del *while* se debe liberar la SC para que otro proceso pueda acceder a ella y darse cuenta de que debe terminar su procesamiento.

```
int cant = 0;
sem mutex = 1;

Process Chico[id: 0..C-1]
{ P(mutex);
  while (cant < N)
  { -- tomar caramelo
    cant = cant + 1;
    V(mutex);
    -- comer caramelo
    P(mutex);
  }
  V(mutex);
}
```



EJERCICIO 3 - modificado

Hay C chicos y hay una bolsa con caramelos limitada a N caramelos **administrada por UNA abuela**. Ella N veces selecciona un chico aleatoriamente y lo deja pasar a tomar un caramelo.

Process Chico[id: 0.. C -1]

```
{ while (haya caramelos)  
    esperar a que la abuela lo llame  
    --tomar caramelo  
    --comer caramelo  
}
```

Como espera a que lo
despierten a él en particular →
usar semáforos privados
espera_chico[C]

Process Abuela

```
{ int i;  
  for i = 1.. $N$   
    { selecciona chico ID  
      despierta al chico ID  
    }  
  avisa que no hay mas caramelos  
}
```

Usar variable
booleana
seguir



EJERCICIO 3 - modificado

```
bool seguir = true;  
espera_chicos[C] = ([C] 0);
```

Process Chico[id: 0..C-1]

```
{ int i;  
  P(espera_chicos[id]);  
  while (seguir)  
  { --tomar caramelo  
    --comer caramelo  
    P(espera_chicos[id]);  
  }  
}
```

Process Abuela

```
{ int i, aux;  
  for i = 1..N  
  { aux = (rand mod C);  
    V(espera_chicos[aux]);  
  }  
  seguir = false;  
}
```

Como se asegura la abuela que no hay más de dos chicos a la vez *tomando caramelo*. Puede despertar a uno cuando el anterior aún no ha tomado el caramelo → Se debe sincronizar tanto el inicio como el final de la interacción con otro semáforo.

EJERCICIO 3 - modificado

```
bool seguir = true;  
sem espera_chicos[C] = ([C] 0), listo = 0;
```

Process Chico[id: 0..C-1]

```
{ int i;  
  
P(espera_chicos[id]);  
while (seguir)  
{ --tomar caramelo  
  V(listo);  
  --comer caramelo  
  P(espera_chicos[id]);  
}  
}
```

Process Abuela

```
{ int i, aux;  
  
for i = 1..N  
{ aux = (rand mod C);  
  V(espera_chicos[aux]);  
  P(listo);  
}  
seguir = false;  
}
```

Como se enteran los chicos que se modificó el valor de *seguir* → después de modificar el valor de *seguir* la abuela debe volver a despertar a cada chico para que entren a su SC y detecten este valor.

EJERCICIO 3 - modificado

```
bool seguir = true;  
sem espera_chicos[C] = ([C] 0), listo = 0;
```

Process Chico[id: 0..C-1]

```
{  int i;  
  
  P(espera_chicos[id]);  
  while (seguir)  
  {  --tomar caramelo  
      V(listo);  
      --comer caramelo  
      P(espera_chicos[id]);  
  }  
}
```

Process Abuela

```
{  int i, aux;  
  
  for i = 1..N  
  {  aux = (rand mod C);  
      V(espera_chicos[aux]);  
      P(listo);  
  }  
  seguir = false;  
  for aux = 0..C-1 → V(espera_chicos[aux]);  
}
```



EJERCICIO 3 - modificado

Si ahora volvemos al enunciado original de la explicación (EJERCICIO 3) y decimos que antes de que la abuela comience debe esperar a que todos lleguen.

```
bool seguir = true;
sem espera_chicos[C] = ([C] 0), listo = 0;
```

Process Chico[id: 0..C-1]

```
{ int i;

P(espera_chicos[id]);
while (seguir)
{  --tomar caramelo
   V(listo);
   --comer caramelo
   P(espera_chicos[id]);
}
}
```

Process Abuela

```
{ int i, aux;

for i = 1..N
{ aux = (rand mod C);
  V(espera_chicos[aux]);
  P(listo);
}
seguir = false;
for aux = 0..C-1 → V(espera_chicos[aux]);
}
```

Al tener el proceso abuela además de los chicos podemos hacer una barrera con coordinador (porque ya existe el proceso).

EJERCICIO 3 – enunciado original

Cada Chico le avisa que llegó con la técnica de señalización de eventos, haciendo un V sobre un semáforo, y la abuela debe hacer tantos P como chicos halla.

```
bool seguir = true; sem espera_chicos[C] = ([C] 0), listo = 0;
```

```
sem aviso = 0;
```

Process Chico[id: 0..C-1]

```
{ int i;  
  V(aviso);  
  P(espera_chicos[id]);  
  while (seguir)  
  { --tomar caramelo  
    V(listo);  
    --comer caramelo  
    P(espera_chicos[id]);  
  }  
}
```

Process Abuela

```
{ int i, aux;  
  for i = 1..C { P(aviso); }  
  for i = 1..N  
  { aux = (rand mod C);  
    V(espera_chicos[aux]);  
    P(listo);  
  }  
  seguir = false;  
  for aux = 0..C-1 → V(espera_chicos[aux]);  
}
```

EJERCICIO 4

En una empresa de genética hay N clientes que envían secuencias de ADN para que sean analizadas y esperan los resultados para poder continuar. Para resolver estos análisis la empresa cuenta con 1 servidores que resuelve los pedidos de acuerdo al orden de llegada de los mismos.

Se necesitan los N procesos *Cliente* para enviar los pedidos y recibir los resultados, y el proceso *Servidor* para resolverlos

Process Cliente[id: 0.. N -1]

```
{ while (true)
  { --generar secuencia  $S$ 
    encolar ( $id$ ,  $S$ )
    esperar resultado
  }
}
```

¿Cómo?

Process Servidor

```
{ while (true)
  { recibir pedido ( $id$ ,  $S$ )
    resolver solicitud  $S$ 
    retornar el resultado a  $id$ 
  }
}
```



EJERCICIO 4

Se debe usar una *cola* C compartida donde se encolan los pedidos para mantener el orden. Al ser compartida el *push* y el *pop* se deben hacer con Exclusión Mutua → para eso usaremos el semáforo *mutex*

```
sem mutex = 1;  
cola C;
```

Process Cliente[id: 0..N-1]

```
{ secuencia S;  
  while (true)  
  { --generar secuencia S  
    P(mutex);  
    push(C, (id, S));  
    V(mutex);  
    esperar resultado  
  }  
}
```

Process Servidor

```
{ secuencia sec;  int aux;  
  while (true)  
  { P(mutex);  
    pop(C, (aux, sec));  
    V(mutex);  
    resolver solicitud sec  
    retornar el resultado a aux  
  }  
}
```

¿Y si la cola está vacía?



EJERCICIO 4

No podemos hacer el *pop* sin estar seguros de que hay algo en la cola, sino se puede producir un error → ¿consultamos por el estado de la cola?

```
sem mutex = 1;  
cola C;
```

Process Cliente[id: 0..N-1]

```
{ secuencia S;  
  while (true)  
    { --generar secuencia S  
      P(mutex);  
      push(C, (id, S));  
      V(mutex);  
      esperar resultado  
    }  
}
```

Process Servidor

```
{ secuencia sec; int aux;  
  while (true)  
    { P(mutex);  
      if not (empty(C)) → pop(C, (aux, sec));  
      V(mutex);  
      resolver solicitud sec  
      retornar el resultado a aux  
    }  
}
```

¿Y si está vacía? → se produce **BUSY WAITING**



EJERCICIO 4

Debe quedarse demorado en un semáforo hasta que seguro haya algo en la cola; cuando un cliente se encolo debe avisar por medio de ese semáforo, usado como *contador de recursos*.

```
sem mutex = 1, pedidos = 0;  
cola C;
```

Process Cliente[id: 0..N-1]

```
{ secuencia S;  
  while (true)  
  { --generar secuencia S  
    P(mutex);  
    push(C, (id, S));  
    V(mutex);  
    V(pedidos);  
    esperar resultado  
  }  
}
```

Process Servidor

```
{ secuencia sec; int aux;  
  while (true)  
  { P(pedidos);  
    P(mutex);  
    pop(C, (aux, sec));  
    V(mutex);  
    --resolver solicitud sec  
    retornar el resultado a aux  
  }  
}
```

¿Cómo devolver el resultado al cliente?



EJERCICIO 4

Usaremos un vector *resultados* para poner el resultado para cada cliente, y un semáforo privado *espera* para cada uno con el cual se le avisa que ya está la respuesta en su posición del vector.

```
sem mutex = 1, pedidos = 0, espera[N] = ([N] 0);  
int resultados[N];  
cola C;
```

Process Cliente[id: 0..N-1]

```
{ secuencia S;  
  while (true)  
    { --generar secuencia S  
      P(mutex);  
      push(C, (id, S));  
      V(mutex);  
      V(pedidos);  
      P(espera[id]);  
      --ver resultado de resultados[id]  
    }  
}
```

Process Servidor

```
{ secuencia sec; int aux;  
  while (true)  
    { P(pedidos);  
      P(mutex);  
      pop(C, (aux, sec));  
      V(mutex);  
      resultados[aux] = resolver(sec);  
      V(espera[aux]);  
    }  
}
```

